Policy-Based Authorization (PBAC) provides a unified approach to these methods under a single framework. It externalizes access logic—moving it out of application code—and evaluates policies at runtime using current information. In a PBAC model, policies serve as the primary decision-making units. Some are written as rules with attributes or conditions (like in ABAC). Others reference roles or organizational structures (as in RBAC). Still others determine access based on relationships between people and resources (as in ReBAC). PBAC treats these as inputs rather than as separate silos.

This chapter has examined each of these models individually. PBAC expands on them, as shown in table 6.6.

Table 6.6 How PBAC incorporates and extends RBAC, ABAC, and ReBAC by treating roles, attributes, and relationships as inputs to externalized policies evaluated at runtime

Model	Core Concept	How PBAC Incorporates It
RBAC	Roles grant permissions	Roles used as input to policy decisions
ReBAC	Relationships define access	Policies reference relationships like ownership or management
ABAC	Attributes drive decisions	Policies written using combinations of attributes

In PBAC, roles, attributes, and relationships are all just data, context that policies can reference dynamically. The core idea is this: *Don't hardcode access decisions; instead, define and evaluate policies as part of a shared, external system.*

Each represents a different style of externalized decision-making. In the next two subsections, we explore both approaches, showing how PBAC provides the flexibility to unify authorization across different domains and use cases.

6.3.1 Policy as data

In section 6.1, we saw how ReBAC expresses policies through relationship data. The rules in a ReBAC system don't explicitly specify every principal or resource—instead, they refer to patterns in the graph. For example, a policy might allow access if the principal is the owner of the resource or if they manage someone who owns it. Another might grant access if the